

**PROJET TUTEURÉ : SLICE
SLIDING CONTACT INVESTIGATION FOR CARBON
PANTOGRAPH WEAR EVALUATION**

**Naël CROSNIER
Théliau FOULER
Paul MITTLER
Nysteroye Christ ZEBAZE NGUENA**

Université Marie et Louis Pasteur – UFR STGI Belfort

Remerciements :

Nous tenons tout d'abord à remercier notre tuteur pédagogique pour ce projet, Philippe BAUCOUR pour son accompagnement tout au long de l'année, ses idées ainsi que ses précieux conseils ayant contribué à l'amélioration de notre maquette.

Nous souhaitons également remercier Cyrille VERNA pour sa disponibilité, son regard avisé sur l'ingénierie de la maquette, ses connaissances physiques et son implication sur l'impression 3D des pièces.

Nous exprimons notre plus grande gratitude envers Oreen OUMER pour le temps qu'il nous a consacré afin de nous présenter son banc d'essai et ses recherches ainsi que pour avoir répondu à nos nombreuses questions sur un sujet complexe.

Les cours de recherche documentaire donnés par Véronique NEYER et Caroline KIMMICH nous ont grandement aidés pour la recherche de sources en lien avec notre sujet, ainsi que dans la construction et la rédaction d'une bibliographie.

Enfin, nous adressons un dernier remerciement à Théo KZYAZIK qui, grâce à sa thèse, nous a permis de compléter notre travail avec des éléments scientifiques pertinents et approfondis.

TABLE DES MATIERES

Présentation du sujet	p.
I. Etude théorique	p.
II. Amélioration de la maquette	p.
2.1. Elements physiques	p.
2.2. Arduino	p.
III. Programmes et gestion des moteurs	p.
3.1. Arduino	p.
3.2. Python	p.
IV. Résultats et Conclusion	p.
V. Références bibliographiques	p.
VI. Annexes	p.
6.1. Programme Arduino	p.
6.2. Programme Python	p.

TABLE DES FIGURES

Figure 1 : Schéma de la liaison Caténaire-Pantographe p.

Figure 2 : Photo de la maquette vue du dessus

Figure 3 : Schéma du “zig-zag” de la caténaire entre les poteaux p.

Figure 4 : Représentation des grandeurs cinématiques d'un point en mouvement circulaire

Figure 5 : Photo de la maquette montrant le placement et les pièces entraînées par la courroie

Présentation du sujet :

Les trains font partie des moyens de transport les plus fréquentés en France (environ 11% de la distance totale parcourue par des voyageurs en France en 2023 d'après [touteleurope.eu](https://www.touteleurope.eu)), notamment dû à ses nombreux avantages comme sa sûreté, sa praticité ou bien son faible impact écologique. A l'heure actuelle, de nombreuses questions se posent à propos des trains, que ce que cela soit au niveau sécurité, prix ou encore énergétique. C'est de ce dernier aspect que nous allons plus particulièrement parler.

En effet, aujourd'hui, les trains utilisent un système nommé caténaire-pantographe pour s'alimenter en énergie (ici ce sera de l'énergie électrique).



Figure 1 : Schéma de la liaison Caténaire-Pantographe.

Ce système est constitué d'une caténaire qui est un ensemble de câbles porteurs servant à acheminer l'électricité jusqu'au train. Cette dernière est transmise au train par le pantographe qui est un dispositif articulé formé d'une suite de bras ainsi que d'un archet sur lequel est fixée une bande de frottement. C'est ici que l'électricité va se transmettre au train.

C'est le contact entre caténaire et pantographe qui permet l'alimentation du train. Cette technologie permet un encombrement moindre par rapport aux anciennes technologies comme les locomotives à charbon et à vapeur mais possède un inconvénient de taille : le frottement. En effet, le frottement entre la bande du pantographe avec la caténaire use rapidement la bande sous l'action de la vitesse (plus ou moins grande suivant le type de train) ou encore la présence d'électricité. Cette usure, principalement sur la bande, est l'élément qui a poussé la SNCF en association avec FEMTO ST à travailler sur cette technologie en étudiant comment réduire cette usure et permettre une maintenance moins soutenue.

Pour bien comprendre la situation, nous avons commencé par récupérer la maquette conçue par le groupe affecté à ce même projet l'année dernière sous la tutelle de Philippe Baucour. Cette maquette est constituée d'un disque tournant ainsi que d'un pion (ici représenté par un crayon) dont on contrôlera la rotation ainsi que les déplacements grâce à l'utilisation de moteurs pas à pas et d'un microcontrôleur. Vous trouverez une photo de cette maquette ci-dessous.

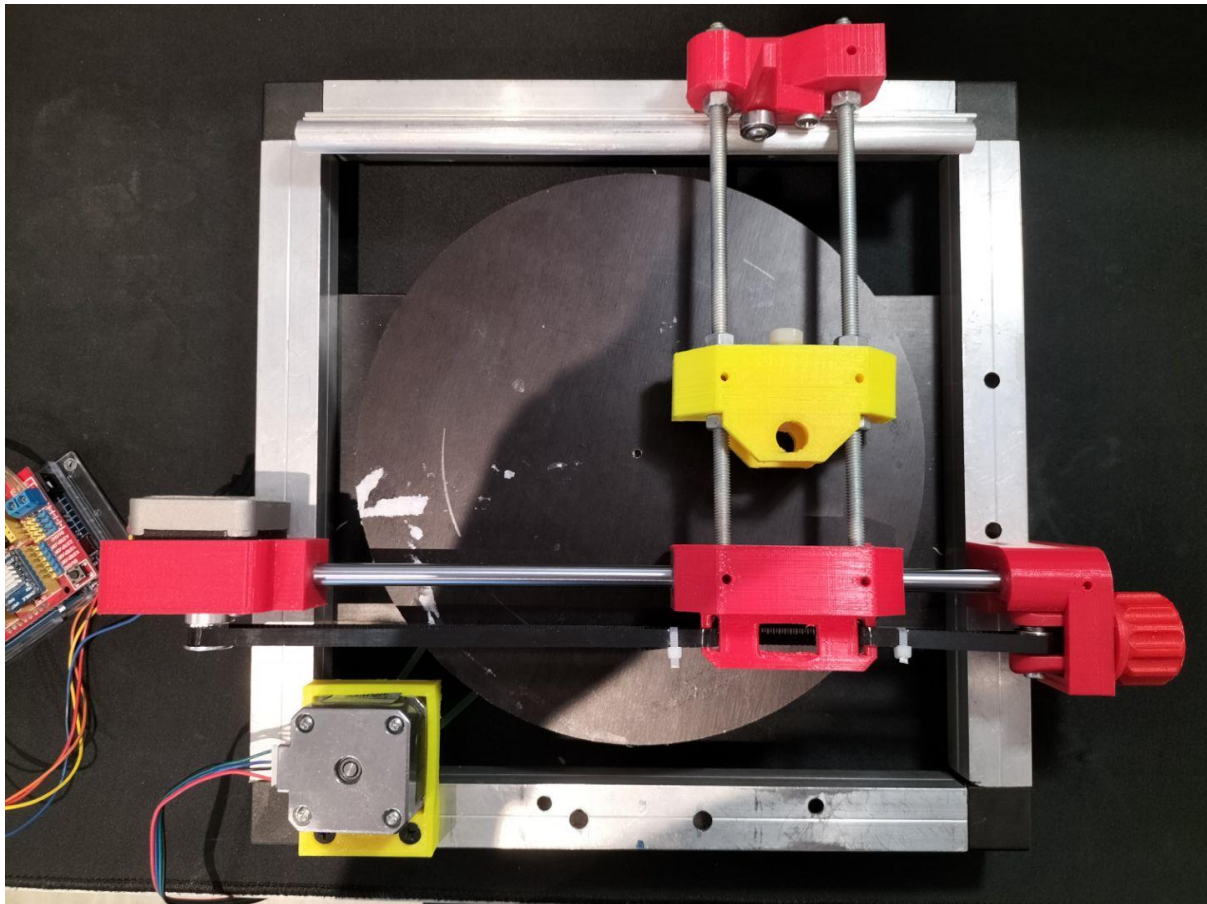


Figure 2 : Photo de la maquette vue du dessus

Cette maquette est directement inspirée du banc expérimental se trouvant à FEMTO ST. Nous avons justement pu voir ce banc en action et recevoir des explications concrètes sur son fonctionnement lors d'essai effectué par le thésard en charge du projet Oreen OUMER.

Le principe de ce banc est de représenter la vitesse du train (que l'on veut idéalement garder constante) par une vitesse de rotation (celle du disque précédemment cité) pour simuler l'avancement du train. Ce dernier sera quant à lui réduit au point de contact entre le caténaire et le pantographe représenté par un pion en carbone qui va venir se frotter sur le disque lorsque ce dernier tournera. Ce point de contact est déplaçable où l'on veut le long du rayon du disque grâce à des moteurs. Le déplacement le long du rayon sert quant à lui à prendre en compte le mouvement du point de contact entre le caténaire et le pantographe qui glisse le long de la bande de contact.

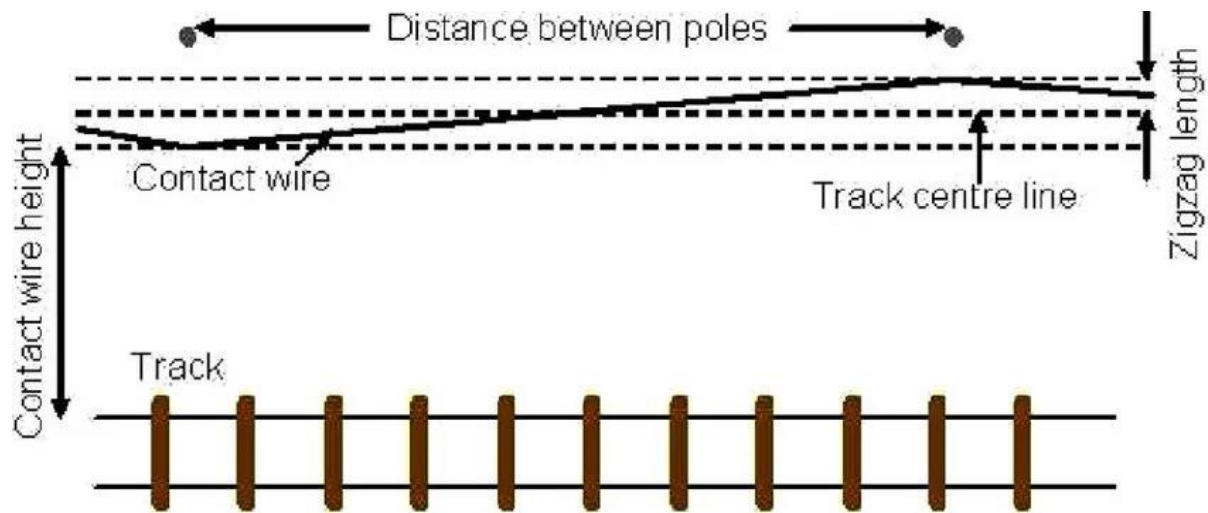


Figure 3 : Schéma du "zig-zag" de la caténaire entre les poteaux.

Ce mouvement du point est pensé pour réduire l'usure de la bande. En faisant varier la position du point de contact, la bande s'use sur toute sa longueur plutôt que sur un seul point ce qui permet d'augmenter sa durée de vie et de réduire ses risques de sciage.

Une fois cette maquette récupérée, nous avons donc pu commencer l'amélioration de cette dernière que cela soit au niveau de la maquette en elle-même (pièce, dimension, etc) mais surtout sur un autre aspect très important de ce projet qui est le contrôle des moteurs. Pour reproduire au mieux la réalité, la gestion des moteurs est cruciale. Ces moteurs sont contrôlés par un microcontrôleur géré en langage Arduino. Ce dernier nous permet de contrôler les moteurs à notre guise en changeant par exemple leur nombre de pas par seconde, donc leurs vitesses de rotation. La bonne gestion de ces moteurs passe aussi par une étude théorique des mouvements et vitesses des différents éléments de la maquette. Avec ces résultats, nous pouvons, une fois les avoir implantés dans le code Arduino, gérer le plus fidèlement possible à la réalité les moteurs.

Ce projet, étant destiné à être présenté au festival INOUIH, il doit pouvoir être compréhensible par le plus grand nombre. Cela nous amène donc à réfléchir sur comment vulgariser ce projet, les thèmes abordés et le fonctionnement de la maquette. Pour cela, en plus d'explications orales pour expliquer les principes fondamentaux, nous avons aussi cherché un moyen de montrer nos résultats et de les représenter de manière ludique et la plus compréhensible possible. Pour cela, nous utiliserons un programme python relié à un écran nous permettant de montrer l'influence des différents paramètres sur les résultats. Cette simulation se devra d'être le plus fidèle possible à la réalité, c'est pourquoi il faudra aussi y implémenter les différentes équations obtenues après l'étude théorique.

Comme nous avons pu le voir, ce projet possède plusieurs aspects très différents nécessaires au bon déroulement de ce dernier. Nous avons donc sectionné notre travail pour voir dans quel ordre les différentes tâches doivent être réalisées.

I) Etude théorique :

La première étape de notre travail a été l'étude théorique du fonctionnement mécanique de la maquette. L'objectif étant d'en tirer des équations permettant le bon fonctionnement des programmes de gestion des moteurs. C'est donc une étape très importante. Pour commencer cela, nous partons d'une équation basique très souvent utilisée lors de problème avec des objets en rotation :

$$V_{tan} = \omega \cdot r$$

D'après cette équation, on a :

- V_{tan} (en m/s, qui est la vitesse tangentielle du train donc l'équivalent de sa vitesse réelle, censée être constante idéalement)
- ω (En rad/s) qui est la vitesse de rotation du disque
- r (en mm) étant le rayon du cercle définit par le point de contact entre le disque et le pion

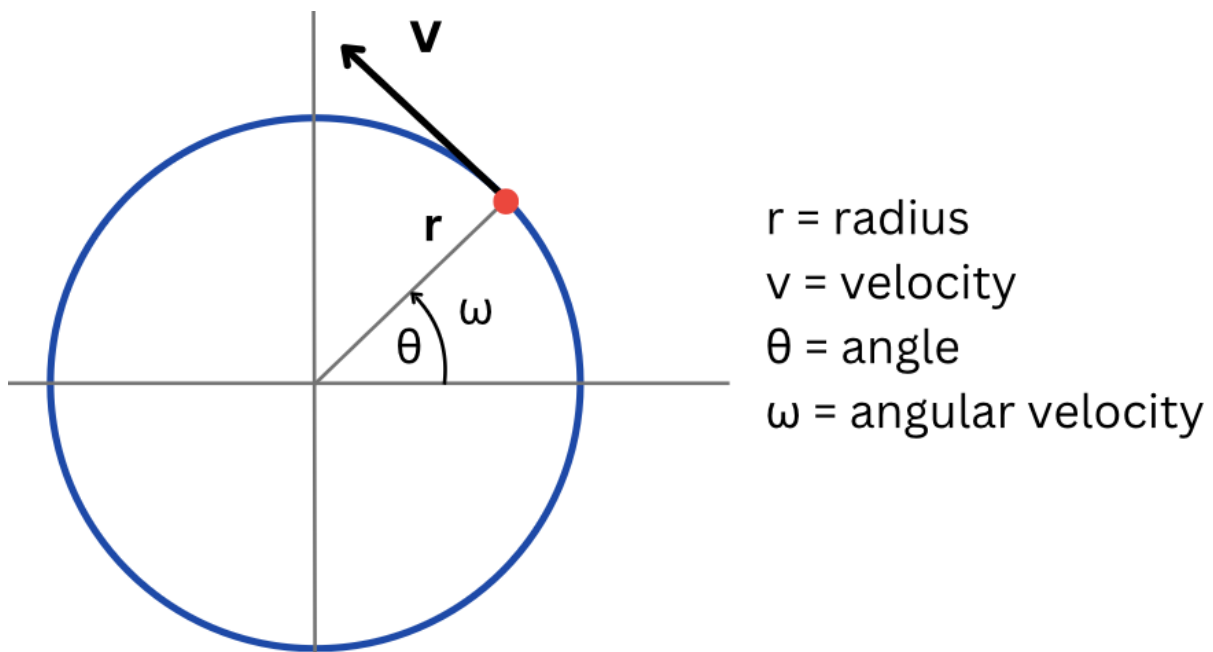


Figure 4 : Représentation des grandeurs cinématiques d'un point en mouvement circulaire.

Cette équation met donc en relation directe la vitesse tangentielle du train et la vitesse de rotation du disque ce qui nous permet de relier nos deux moteurs.

Notre base étant établie, nous allons maintenant utiliser cette équation pour l'adapter à notre situation. Comme nous le savons, nous avons 2 moteurs sur la maquette, chacun entraînant

un élément différent et nécessitant donc une étude différente. Nous nommerons le moteur m_1 (moteur au support jaune) celui qui gère la rotation du disque et le moteur m_2 (moteur au support rouge) celui qui gère le déplacement du pion sur le disque.

Pour mieux comprendre ce dont on parle, voici ci-dessous une photo de la maquette avec les différents éléments de cette dernière nommés :

L'objectif ici est de recalculer la vitesse tangentielle en direct pour s'assurer qu'elle reste constante. Les programmes seront chargés de calculer ω en fonction de r car le calcul de r quant à lui dépendra d'une autre relation qui est : $d_p = \frac{N \cdot R_p}{360}$

Où N est le nombre de pas effectué par le moteur, R_p est le rayon de la poulie permettant le mouvement du pion le long du rayon du disque ($R_p = 70 \text{ mm}$)

Ce d_p est quant à lui la distance parcourue par le pion le long du rayon du disque. Cette distance sera ajoutée à notre ancien rayon r pour pouvoir le recalculer. Avec cette définition de r , cela nous permet de calculer ω en fonction d'un rayon calculable facilement et d'une vitesse tangentielle constante. Le nombre de pas se détermine expérimentalement pour que la gestion des moteurs soit la plus fluide possible mais tout cela se fera avec les différents programmes informatiques.

Grâce à l'aide de Monsieur Verna nous pu obtenir l'équation suivante :

$$V_{tan} = \omega_1 \cdot r_1 = \omega_2 \cdot r_2$$

Ici ω_1 et ω_2 sont les vitesses de rotation du disque à différents rayons r_1 et r_2 .

Avec ces différentes informations et équations, nous pouvons donc calculer ω en fonction de r qui lui est connu. Cela nous permet que, lorsque l'on fera varier le nombre de pas, ω varie lui aussi pour garder la vitesse tangentielle constante.

II) Amélioration de la maquette :

i. Éléments physiques

Une fois la maquette obtenue, nous avons commencé par tenter de la faire fonctionner pour voir s'il y avait d'éventuels problèmes. En effet, une fois la maquette active, des problèmes nous sont apparus comme principalement le fait que le disque n'est pas stable une fois qu'il est en rotation ou encore qu'il tourne de manière saccadée et peu précise. Nous avons donc compris que des améliorations de la maquette s'imposait. C'est ce dont on va parler dans cette partie.

Pour commencer, nous avons cherché à régler le problème de stabilité du plateau. Pour cela, nous avons vu avec M. VERNA et M. BAUCOUR pour modéliser une pièce 3D afin de mieux y fixer le plateau et ainsi éviter tout oscillement. Une première idée était d'entourer le moyeu central et de former une sorte de plateau pour y fixer le disque mais finalement l'essai s'est avéré non concluant. Une seconde idée était une pièce se mettant autour du moyeu mais s'insérant dans le plateau afin d'y mettre une vis et une vis oreille afin de pouvoir démonter le plateau de façon rapide et simple. Après prise de mesure, modélisation 3D sur Onshape, un premier essai non fonctionnel à cause d'erreurs de mesures, nous avons enfin eu la pièce finale réglant parfaitement le problème.

ii. Arduino

Le second point d'amélioration est la partie programmation dans le langage Arduino, nous asservissons les moteurs par le biais d'une carte Arduino, dans laquelle nous téléverrons du code. L'un des objectifs majeurs de cette partie était de programmer un script qui était conforme aux équations mécaniques et cinématiques régissant les mouvements du système plateau-pion.

Initialement confié à une carte Arduino Nano (équipée d'un microcontrôleur ATmega328P), le système souffrait d'un goulot d'étranglement calculatoire sévère. Le maintien de la consigne avec notre rapport de transmission exigeait la génération d'environ 13 300 impulsions par seconde. L'architecture 8-bits de la Nano, cadencée à 16 MHz et dépourvue d'unité de calcul en virgule flottante (FPU), devait émuler les opérations non linéaires complexes de la boucle d'asservissement (divisions, variables de type *float*, calculs impliquant π) de manière purement logicielle. Cela entraînait une saturation de l'horloge processeur autour de 4 000 pas/s et un flouage cinématique de la maquette.

La migration vers une carte Arduino UNO R4 Minima a permis de lever cette limite matérielle. Embarquant un processeur ARM Cortex-M4 (32-bits) cadencé à 48 MHz, cette carte intègre une FPU matérielle. Cette architecture permet d'exécuter les opérations en virgule flottante en un nombre très réduit de cycles d'horloge. La boucle logicielle peut désormais générer des fréquences dépassant les 20 000 pas/s sans induire de latence.

L'interface de puissance entre la logique 5V du microcontrôleur et l'alimentation 8V des moteurs pas-à-pas est assurée par un Shield CNC V3 équipé de drivers de type A4988.

Un moteur pas-à-pas standard effectue 200 pas entiers par tour (soit une résolution angulaire de 1,8° par pas). En mode "Full Step", chaque transition de phase génère des transitoires de couple et des résonances mécaniques nuisibles à l'étude tribologique du contact glissant, qui requiert une continuité parfaite.

Afin de lisser la dynamique du mouvement, nous avons modifié la configuration matérielle de la carte de commande pour forcer un fonctionnement en Microstepping.

Tableau : Configuration des états logiques du driver A4988 (Microstepping 1/8)

Paires de broches	État Logique cible	Action matérielle réalisée
M0 (ou MS1)	HIGH (1)	Pont de court-circuit effectué (soudage)
M1 (ou MS2)	HIGH (1)	Pont de court-circuit effectué (soudage)
M2 (ou MS3)	LOW (0)	Contact laissé ouvert

Cette subdivision électronique des courants de phase règle l'intensité sous forme de signaux pseudo-sinusoïdaux, portant la résolution à 1 600 micro-pas par tour. Le mouvement linéaire du pion est ainsi devenu strictement continu, limitant l'excitation des modes propres de la structure.

L'affranchissement des limites logicielles de traitement a mis en exergue une contrainte purement électromagnétique de l'actionneur. Lors du passage du pion dans la zone centrale du disque, la consigne haute fréquence du microcontrôleur provoquait une perte de synchronisme et un décrochage brutal du moteur du plateau (phénomène observé autour de 11 000 pas/s, soit environ 412 Tr/min).

Ce comportement est gouverné par la Force Contre-Électromotrice (FCEM) et l'inductance (L) des enroulements du moteur. La dynamique de montée du courant dans la bobine est régie par l'équation de maille temporelle : $\frac{di}{dt} = \frac{U-E}{L}$ (où E est la FCEM induite par la rotation du rotor).

À haute vitesse angulaire, l'inversion rapide du champ et la forte valeur de la FCEM s'opposent à l'établissement du courant. Sous notre tension d'alimentation contrainte à 8V, le gradient de courant $\frac{di}{dt}$ est d'autant plus limité : le courant n'a physiquement plus le temps d'atteindre sa valeur nominale avant la commutation de phase suivante. Le couple moteur s'effondre drastiquement, ne parvenant plus à vaincre l'inertie du disque.

Pour pallier cette limitation physique directement imputable à l'alimentation de 8V, un écrêtage par saturation logicielle (limite fixée à 10 000 pas/s) a été implémenté. Ce compromis garantit la stabilité électromagnétique du système en boucle ouverte en bornant le fonctionnement à un domaine cinématique viable.

Un élément important de la maquette est la courroie. Elle sert à transmettre le mouvement créer par le moteur m_1 à l'axe sur lequel le disque se fixe pour pouvoir tourner.

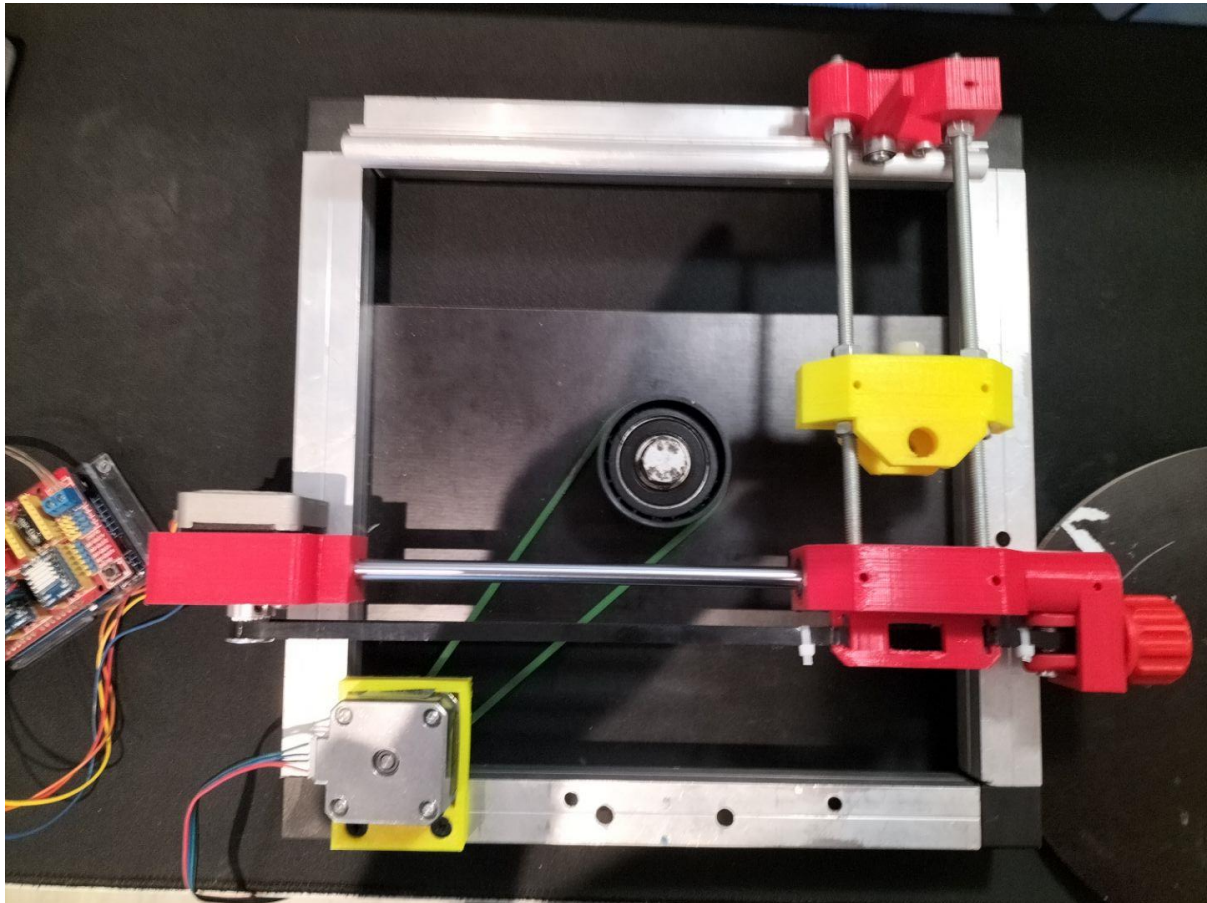


Figure 5 : Photo de la maquette montrant le placement et les pièces entraînées par la courroie

Cette dernière n'était plus assez tendue donc Monsieur Verna nous en a refait une avec les mêmes dimensions pour nous assurer une plus grande fiabilité et une meilleure précision.

Cette maquette doit permettre de visualiser nos résultats c'est pourquoi nous avons décidé de réfléchir à comment améliorer cette visualisation. Nous avons donc décidé de changer le support servant à représenter le point de contact entre le pion et le disque. Avant, ce pion était modélisé par un crayon que l'on pose sur un support pour que la trajectoire du pion se dessine sur une feuille de papier collée sur le disque quand ce dernier tourne. Cela posait des problèmes de frottements car pour avoir un trait visible, il fallait appuyer le crayon assez fort ce qui n'était pas pratique. De plus, le mouvement saccadé du disque dû aux problèmes précédemment cités, entraînait une trajectoire dessinée peu précise. Pour changer cela, le précédent groupe avait donc penser à utiliser un revêtement de type tableau blanc à coller sur le disque pour que le crayon remplacé par un feutre, puisse plus simplement dessiner la trajectoire du pion en réduisant l'influence des frottements sur les résultats obtenus. Nous

avons donc repris cette idée car cela permet aussi de pouvoir effacer la trajectoire dessinée plus facilement qu'avec du papier classique.

D'autres problèmes plus légers sont aussi à régler comme :

- la fixation des moteurs sur leur support, notamment le moteur m_2 qui lorsque la maquette est active, n'était pas très stable. Ce problème détecté par Monsieur Verna lors d'un entretien, a tout de suite été corrigé après avoir fixé ces moteurs aux supports avec des vis.
- la stabilisation générale de la maquette et la diminution des vibrations. Pour cela, nous avons placés des patins anti-dérapants que l'on a collé sous les pieds de la maquette pour réduire les vibrations qu'elle subissait quand elle était active.

Comme nous avons pu le voir, la structure et le fonctionnement général de la maquette n'a pas été modifié. Notre travail se situe surtout la stabilisation des éléments de la maquette comme le disque, la bonne transmission des informations et mouvements avec la gestion des équipements du micro-processeur et de la courroie ou encore améliorer la visibilité de nos résultats.

III) Programmes et gestion de moteurs :

Pour pouvoir mener à bien nos tests et expériences, il nous faut utiliser des programmes informatiques pour pouvoir gérer les moteurs et afficher nos résultats afin de mieux comprendre ce qu'il se passe. Nous allons donc utiliser deux programmes différents : le premier est un code Arduino qui, à l'aide du microcontrôleur de la carte Arduino, nous permettras de contrôler les moteurs de la manière la plus précise voulue afin d'obtenir les meilleurs résultats possibles. Le deuxième est un programme Python qui sert quant à lui à afficher nos résultats de manière visuelle pour simplifier la compréhension du sujet à un public non initié.

Nous allons commencer par aborder le programme Arduino car il est nécessaire au bon fonctionnement de la maquette.

iii. Arduino

Contrairement au code Python dont l'objectif principal est la simulation et la visualisation des données, le programme Arduino constitue le cœur opérationnel de notre maquette. Son rôle est de traduire les résultats de notre étude cinématique théorique en signaux électriques concrets afin de piloter les moteurs pas-à-pas en temps réel. Pour garantir une gestion précise des signaux, ce programme s'appuie sur la bibliothèque externe `AccelStepper` et a été rigoureusement structuré en trois blocs distincts.

1. Initialisation et Variables globales

Ce premier bloc prépare l'environnement matériel et logiciel de la carte (Arduino UNO R4 Minima). Il définit d'abord le câblage du microcontrôleur vers le Shield CNC, puis modélise les caractéristiques mécaniques de la maquette, notamment l'activation du Microstepping (réglé sur 1/8 de pas pour réduire les vibrations) et l'intégration de notre rapport de transmission (*multiplicateur_mecanique*). C'est également ici que sont déclarées les constantes physiques fondamentales au cœur de l'étude, à savoir la vitesse tangentielle cible (25 cm/s) et les limites spatiales du disque. Enfin, pour protéger l'intégrité du matériel, des garde-fous logiciels (écrêtage de la vitesse maximale) sont fixés afin d'éviter le décrochage électromagnétique du moteur face aux très hautes fréquences d'impulsion.

2. Boucle principale

Ce bloc, s'exécutant en continu à haute fréquence, constitue le cœur de notre asservissement. Il orchestre deux processus cinématiques simultanés. D'une part, il gère le balayage linéaire du pion en contrôlant ses allers-retours autonomes entre le bord et le centre du disque. D'autre part, il assure la régulation dynamique de la vitesse du plateau. Afin d'optimiser la puissance de calcul du processeur et d'éviter les saccades, le recalcul de la vitesse n'est pas effectué à chaque micro-pas, mais selon un échantillonnage spatial défini (*SEUIL_PAS_Y*). À chaque palier, le programme déduit le rayon actuel et applique l'équation cinématique de base ($\omega =$

$\frac{v_{tan}}{r}$) pour ajuster instantanément la vitesse de rotation du disque. L'écrtage de sécurité (*constrain*) garantit que si la vitesse angulaire requise tend vers l'infini près du centre du disque, elle est automatiquement bridée au plafond toléré par le moteur.

3. Fonctions mathématiques

Le dernier bloc regroupe les outils de conversion essentiels au système. L'architecture de la maquette fonctionnant en boucle ouverte (sans capteur de position absolu), l'Arduino est initialement aveugle à la position physique du pion. Ces fonctions permettent de lier les variables informatiques à la réalité mécanique. Elles convertissent les grandeurs mécaniques (RPM) en signaux électriques temporels (pas par seconde), et surtout, elles convertissent le comptage logiciel des pas du moteur en une véritable coordonnée spatiale géométrique (le rayon actuel en centimètres), donnée indispensable à la résolution de l'équation dans la boucle principale.

iv. Python

Nous allons maintenant nous pencher sur le code Python. Ce dernier n'est pas nécessaire au fonctionnement de la maquette car il ne commande pas les moteurs. Son but est de visualiser les résultats de manière ludique et compréhensible. Pour cela, nous avons utilisé une bibliothèque appelée Pygame. C'est un ensemble de module nous permettant de créer des simulations comme celle utilisée ou bien même des jeux comme nous avons pu le faire lors d'un cours du premier semestre. Cette bibliothèque est très utile car elle nous permet de structurer le code de la simulation comme on le souhaite.

En effet, un code bien construit permet une exploration et utilisation plus simple de celui-ci. C'est pourquoi nous avons arrangé notre code en plusieurs blocs ayant tous une fonction propre.

Le premier bloc sert à initialiser donc définir toutes les variables dont on a besoin ainsi qu'importer toutes les bibliothèques nécessaires dont Pygame. Les autres bibliothèques servent quant à elles à nous permettre d'utiliser des fonctions mathématiques ou encore de faciliter la gestion des objets en mouvements. Ce bloc est la base du code, il est donc important de définir toutes les variables dont nous avons besoin pour plus tard. Ces variables peuvent être les différentes grandeurs physiques comme des vitesses ou des rayons ou bien encore des éléments de la simulation comme la taille de l'écran ou bien les couleurs de ce dernier.

Le deuxième bloc sert à définir les fonctions. Ces fonctions transforment et utilisent les variables définies avant pour effectuer des calculs comme ceux de la vitesse tangentielle, tracer des formes comme le pion que l'on va faire bouger. Ces fonctions forment la structure de la simulation. Ce sont elles qui vont définir son apparence, comment elle va traiter les données et à quoi elle va servir. Elles sont en quelques sortes le squelette de la simulation. Il est donc logique de les séparer de l'initialisation des variables (qui est naturellement avant les fonctions) et de la suite du code qui sert encore à une autre chose.

Le dernier bloc est celui de la boucle principale. Cette boucle est le cœur du programme. Elle commence dès que l'utilisateur lance le programme et s'arrête lorsque l'utilisateur le ferme. Elle sert à actualiser les données et à les afficher en temps réel. Elle gère la gestion des entrées donc les touches utilisées par l'utilisateur, met à jour les résultats des calculs et rafraichît l'affichage de la simulation à chaque fois que la boucle est effectuée. Cette boucle se servant des fonctions et des variables, il est logique de la placer à la toute fin de ce dernier pour lui permettre d'utiliser tout ce dont elle a besoin.

L'objectif de séparer le code en plusieurs blocs est double : trouver plus simplement les lignes et les parties que l'on veut et permettre une meilleure compréhension de la logique d'un code et du fonctionnement de la simulation.

IV) Résultats et Conclusion :

En conclusion, les développements matériels et logiciels apportés à la maquette initiale ont permis de répondre pleinement aux exigences de ce banc d'essai cinématique. Comme l'a validé notre simulation Python en amont, l'implémentation de notre modèle mathématique garantit le maintien d'une vitesse tangentielle constante en dépit de la variation continue du rayon. Le défi majeur des saccades mécaniques a été résolu par l'exploitation du microstepping (1/8 de pas), apportant ainsi la fluidité de glissement indispensable à une modélisation de l'usure du pantographe.

Par ailleurs, l'optimisation de ce système ne s'est pas limitée à l'aspect algorithmique. La conception et l'impression 3D de nouvelles pièces de maintien, couplées à l'ajout de patins amortisseurs, ont considérablement réduit les résonances et stabilisé le disque en rotation.

D'un point de vue gestion de projet, ce travail a mis en lumière l'importance d'une planification rigoureuse. L'organisation initiale s'est révélée sous-optimale face à la complexité du travail prévue, entraînant des retards notables sur notre calendrier prévisionnel. Toutefois, cette confrontation à la réalité expérimentale nous a poussés à réévaluer nos méthodes de travail collaboratif. Cette capacité d'adaptation nous a permis de surmonter ces obstacles et d'atteindre, *in fine*, l'ensemble des objectifs techniques fixés.

Ce projet nous a ainsi permis de consolider une approche d'ingénierie multidisciplinaire, alliant la modélisation théorique, la programmation embarquée et la conception mécanique. La maquette est aujourd'hui fonctionnelle, stabilisée et fidèle à la réalité physique.

V) Bibliographie :

[<https://ecampusontario.pressbooks.pub/>] (figure 3)

https://www.researchgate.net/figure/Layout-of-an-overhead-electrified-railway-showing-the-zigzag-of-the-contact-wire-adapted_fig3_224219101

<https://actgv.fr/wp-content/uploads/2014/04/cat%C3%A9naire-et-fils-de-contact.pdf>

- Enrici P, Martiré T, Coillot C, Huselstein JJ. Maquette analogique pour l'étude de boucle de régulation. Application à la régulation d'un Moteur à Courant Continu. Journal sur l'enseignement des sciences et technologies de l'information et des systèmes. 2023;22:1015.
- Janssoone D. Manuel d'exploitation ferroviaire: formation des exploitations du ferroviaire, IUT, CNAM, écoles d'ingénieurs. Malakoff: Dunod; 2024.
- Bausseron T. Etude de l'échauffement de la caténaire lors du captage à l'arrêt: développement d'un outil informatique [Thèse de doctorat]. Besançon: Université de Franche-Comté; 2014.
- Jonckheere B. Caractérisation multi-physique d'une interface multi-contacts à l'arrêt et en mouvement: cas du captage de courant via l'interface caténaire-pantographe [Thèse de doctorat en ligne]. Amiens: Université de Picardie Jules Verne; 2021 [cité 13 novembre 2025]. Disponible sur: <https://theses.hal.science/tel-03828928>
- Cincou A, Fortin C. Guide de modélisation 3D: document de référence. Montréal: Presses Internationales Polytechnique; 2019.
- Kziazzyk T. Etude expérimentale de l'usure et de l'échauffement d'un contact électrique glissant: application caténaire-pantographe [Thèse de doctorat en ligne]. Besançon: Université de Bourgogne Franche-Comté; 2023 [cité 06 novembre 2025]. Disponible sur: https://ariane.unlp.fr/discovery/fulldisplay?docid=alma991006709719506657&context=L&vid=33UFC_INST:33UFC_INST&lang=fr&search_scope=MyInst_and_CI&adaptor=Local%20Search%20Engine&tab=Everything&query=any,contains,Etude%20exp%C3%A9rimentale%20de%20l%27usure%20et%20de%20l%27%C3%A9chauffement%20d%27un%20contact%20%C3%A9lectrique%20glissant%20:%20application%20cat%C3%A9naire-pantographe&offset=0
- Delcey N. Modélisation électro-thermique d'un pantographe pour un train en mouvement [Thèse de doctorat en ligne]. Besançon: Université de Bourgogne Franche-comté; 2018 [cité 05/12/2025] Disponible sur : https://ariane.unlp.fr/discovery/fulldisplay?docid=alma991006234119706657&context=L&vid=33UFC_INST:33UFC_INST&lang=fr&search_scope=MyInst_and_CI&adaptor=Local%20Search%20Engine&tab=Everything&query=any,contains,contact%20electrique%20pour%20les%20trains&offset=0
- Sreejyothi KR, Chenchireddy K, Srujana A, Nagaraju D, Ramcharan G, Raghu D. Arduino based stepper motor speed regulation for robotics applications. International Journal of Applied Power Engineering (IJAPE) [En ligne]. 2024 [cité 05 décembre 2025];13(3):695-702. Disponible sur: <http://doi.org/10.11591/ijape.v13.i3.pp695-702>

VI) Annexe

v. Code Arduino :

```
1 //Maquette de démonstration pour le projet SLICE
2 //Objectif : Contrôler un moteur linéaire (pion) et un moteur rotatif (disque) en
  synchronisation
3 //avec une vitesse tangentielle constante pour le pion.
4 //
5 //Matériel :
6 //- Arduino UNO R4 MINIMA
7 //- Shield CNC pour contrôler les moteurs pas à pas
8 //- Moteur linéaire (pion) avec driver compatible
9 //- Moteur rotatif (disque) avec driver compatible
10 //
11 //Fonctionnement :
12 //- Le moteur Y (pion) effectue un mouvement linéaire aller-retour entre deux
  positions définies.
13 //- Le moteur X (disque) ajuste sa vitesse de rotation en fonction de la position
  du pion pour maintenir une vitesse tangentielle constante.
14 //
15 //Note : Ce code est conçu pour être utilisé en mode FULL STEP pour une meilleure
  précision et un contrôle plus fin des moteurs. Assurez-vous que les drivers de vos
  moteurs sont configurés pour le microstepping approprié (1/8 dans ce cas) pour
  correspondre aux constantes définies dans le code.
16
17 // 1. --- Initialisation et Variables globales ---
18 #include <AccelStepper.h>
19
20 // --- BROCHES DU SHIELD CNC ---
21 #define EN 8 // Broche d'activation du shield
22 #define STEP_Y 3 // Broche STEP du moteur Y (pion)
23 #define DIR_Y 6 // Broche DIR du moteur Y
24 #define STEP_X 2 // Broche STEP du moteur X (plateau/disque)
25 #define DIR_X 5 // Broche DIR du moteur X
26
27 // --- RÉOLUTION MÉCANIQUE (FULL STEP) ---
28 const int MICROSTEPPING = 8; // Microstepping du driver (1/8 de pas)
29
30 const float multiplicateur_mecanique = 2.83;
31 const int stepsPerRevolutionX = (200 * MICROSTEPPING) * multiplicateur_mecanique;
  // Pas par tour du moteur X
32 const int stepsPerRevolutionY = 200 * MICROSTEPPING; // Pas par tour du moteur Y
33
34 // --- CONSTANTES PHYSIQUES ---
35 const float VITESSE_TANGENTIELLE_CIBLE = 25.0; // Vitesse tangentielle du pion en
  cm/s
36 const float vitesseY_RPM = 0.5; // Vitesse de rotation du moteur Y
  en RPM
37 const int pasParCycle = 290 * MICROSTEPPING; // Nombre de pas pour un cycle
  aller-retour du pion
38 const float RAYON_MAX = 7.5; // Rayon maximal du disque en cm
39 const float RAYON_MIN = 1.35; // Rayon minimal du disque en cm
40
41 // --- CONSTANTES DE RÉGLAGE LOGICIEL ---
42 const long SEUIL_PAS_Y = 16; // Seuil de variation en pas pour
  recalculer la vitesse X
43 const float VITESSE_X_MIN = 10.0; // Vitesse minimale autorisée pour le
  plateau X
```

```

44 const float VITESSE_X_MAX = 10000.0;      // Vitesse maximale autorisée pour le
    plateau X
45 const float SEUIL_DIFF_VITESSE_X = 2.0;   // Seuil de variation de vitesse pour
    appliquer un nouveau réglage
46
47 AccelStepper moteurY(AccelStepper::DRIVER, STEP_Y, DIR_Y);
48 AccelStepper moteurX(AccelStepper::DRIVER, STEP_X, DIR_X);
49
50 long positionDepart = 0;                  // Position au début du cycle Y
51 long positionCible = pasParCycle;         // Position cible initiale pour le mouvement Y
52 long dernierPasY = 0;                     // Position Y mémorisée pour limiter les
    recalculs
53 float vitesseX_actuelle = 0.0;           // Vitesse courante appliquée au moteur X
54
55 void setup() {
56     Serial.begin(115200);
57
58     // --- ACTIVATION DU SHIELD ---
59     pinMode(EN, OUTPUT);
60     digitalWrite(EN, LOW);
61
62     // Initialisation Moteur Y (Pion)
63     moteurY.setMaxSpeed(convertRPMTToStepsPerSec(vitesseY_RPM, stepsPerRevolutionY));
64     moteurY.setSpeed(convertRPMTToStepsPerSec(vitesseY_RPM, stepsPerRevolutionY));
65
66     // Initialisation Moteur X (Disque)
67     moteurX.setMaxSpeed(VITESSE_X_MAX);
68     moteurX.setSpeed(100);
69
70     Serial.println("Initialisation SLICE (Mode FULL STEP) OK.");
71 }
72
73 // 2. --- BOUCLE PRINCIPALE ---
74 void loop() {
75     moteurY.runSpeed();
76     moteurX.runSpeed();
77
78     // --- GESTION DU MOTEUR Y (ALLER-RETOUR DU PION) ---
79     // Quand le pion atteint la cible, on inverse sa direction
80     if ((moteurY.currentPosition() >= positionCible && moteurY.speed() > 0) ||
81         (moteurY.currentPosition() <= positionCible && moteurY.speed() < 0)) {
82
83         moteurY.setSpeed(-moteurY.speed());
84         positionDepart = moteurY.currentPosition();
85         // Nouvelle cible dépendante du sens actuel du mouvement
86         positionCible = positionDepart + (pasParCycle * (moteurY.speed() > 0 ? 1 : -
1));
87
88         Serial.print("Inversion Y. Cible : ");
89         Serial.println(positionCible);
90     }
91
92     // --- ASSERVISSEMENT DU MOTEUR X (VITESSE DU DISQUE) ---
93     long positionY = abs(moteurY.currentPosition());
94
95     if (abs(positionY - dernierPasY) >= SEUIL_PAS_Y) {
96         float rayon = mapPasToRayonSymetrique(positionY);
97
98         // Calcul de la vitesse angulaire nécessaire pour garder la vitesse
    tangentielle cible
99         float vitesseX = (VITESSE_TANGENTIELLE_CIBLE * stepsPerRevolutionX) / (2.0 * PI
    * rayon);
100
101         vitesseX = constrain(vitesseX, VITESSE_X_MIN, VITESSE_X_MAX);
102
103         if (abs(vitesseX - vitesseX_actuelle) > SEUIL_DIFF_VITESSE_X) {

```

```

104     moteurX.setSpeed(vitesseX);
105     vitesseX_actuelle = vitesseX;
106
107     Serial.print("Rayon(cm):");
108     Serial.print(rayon);
109     Serial.print("\tVitesse_Plateau(pas/s):");
110     Serial.print(vitesseX);
111     Serial.print("\tPosition_Pion(pas):");
112     Serial.println(positionY);
113 }
114
115     dernierPasY = positionY;
116 }
117 }
118
119 // --- FONCTIONS MATHÉMATIQUES ---
120 float convertRPMToStepsPerSec(float rpm, int stepsPerRev) {
121     return (rpm * stepsPerRev) / 60.0;
122 }
123
124 float mapPasToRayonSymetrique(long pas) {
125     long cycle = pas % (2 * pasParCycle);
126     float phase = (cycle < pasParCycle) ? (float)(pasParCycle - cycle) / pasParCycle
127         : (float)(cycle - pasParCycle) / pasParCycle;
128     return RAYON_MIN + (RAYON_MAX - RAYON_MIN) * phase;
129 }

```

vi. Code Python:

```

import sys          # Permet de quitter proprement le programme
import math         # Permet d'utiliser pi, cos, sin, etc.
import pygame       # Bibliothèque utilisée pour l'affichage graphique

pygame.init()       # Initialisation de Pygame

# =====
# FENÊTRE
# =====
WIDTH, HEIGHT = 1400, 900      # Dimensions de la fenêtre
FPS = 60                      # Nombre d'images par seconde

screen = pygame.display.set_mode((WIDTH, HEIGHT)) # Création de la fenêtre
pygame.display.set_caption("SLICE - Banc expérimental avec courbes") # Titre
clock = pygame.time.Clock()    # Horloge pour contrôler la vitesse

# =====
# COULEURS
# =====
DARK_BG = (6, 8, 18)          # Fond sombre
PANEL_BG = (235, 240, 250)    # Fond clair des panneaux

WHITE = (245, 245, 245)       # Blanc
BLACK = (0, 0, 0)             # Noir
GRAY = (80, 90, 110)          # Gris
YELLOW = (255, 220, 80)       # Jaune

COPPER = (190, 115, 45)       # Couleur cuivre du disque
COPPER_LIGHT = (230, 150, 70) # Cuivre clair
COPPER_DARK = (90, 50, 20)    # Cuivre foncé

CARBON = (45, 45, 45)         # Couleur du pion carbone

BLUE = (40, 120, 255)         # Bleu
RED = (255, 80, 80)           # Rouge

```

```

GREEN = (80, 200, 120)          # Vert

# =====
# POLICES
# =====
FONT = pygame.font.SysFont("consolas", 24)      # Police normale
FONT_SMALL = pygame.font.SysFont("consolas", 18) # Petite police
FONT_BIG = pygame.font.SysFont("arialblack", 36) # Grande police

# =====
# PARAMÈTRES PHYSIQUES DU CODE ARDUINO
# =====
RAYON_MIN_CM = 1.35              # Rayon minimal réel en cm
RAYON_MAX_CM = 7.5              # Rayon maximal réel en cm

V0_CM_S = 25.0                  # Vitesse tangentielle cible en cm/s

# =====
# GÉOMÉTRIE VISUELLE
# =====
CENTER = (400, HEIGHT // 2 + 40) # Centre du disque décalé vers la gauche

R_DISK_PX = 300                  # Rayon graphique du disque en pixels

PX_PAR_CM = R_DISK_PX / RAYON_MAX_CM # Conversion : nombre de pixels pour 1 cm

r_min_px = RAYON_MIN_CM * PX_PAR_CM # Rayon minimal converti en pixels
r_max_px = RAYON_MAX_CM * PX_PAR_CM # Rayon maximal converti en pixels

# =====
# VARIABLES DE SIMULATION
# =====
Nbp = 200                        # Nombre de points utilisé pour régler le déplacement
Rp = 70                          # Paramètre de déplacement du pion

r_px = float(r_min_px)           # Position initiale du pion en pixels
r_dir = 1.0                      # Sens du déplacement du pion

theta = 0.0                      # Angle de rotation du disque
paused = False                   # Variable de pause

# =====
# SURFACES POUR TRACE ET HEATMAP
# =====
trace_surf = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA) # Surface de trace
blanche
heat_surf = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)  # Surface de heatmap
rouge
heat_surf.fill((0, 0, 0, 0)) # Fond transparent

visit = {}                      # Dictionnaire qui compte les passages du pion

# =====
# DONNÉES POUR LES COURBES
# =====
rayon_data = []                 # Liste des rayons en cm
omega_data = []                 # Liste des vitesses angulaires
vt_data = []                    # Liste des vitesses tangentielles

MAX_POINTS = 250                # Nombre maximal de points affichés sur le graphe

# =====
# CONVERSION PIXELS -> CM
# =====
def px_to_cm(val_px):
    return val_px / PX_PAR_CM    # Convertit une distance en pixels vers cm

```

```

# =====
# POSITION DU PION
# =====
def pion_world_pos(r_val):
    cx, cy = CENTER          # Récupération du centre du disque
    return cx + r_val, cy    # Le pion se déplace horizontalement

# =====
# FOND DE L'ÉCRAN
# =====
def draw_background():
    screen.fill(DARK_BG)    # Efface l'écran avec un fond sombre

    for x in range(0, WIDTH, 50): # Lignes verticales de la grille
        pygame.draw.line(screen, (18, 22, 38), (x, 0), (x, HEIGHT), 1)

    for y in range(0, HEIGHT, 50): # Lignes horizontales de la grille
        pygame.draw.line(screen, (18, 22, 38), (0, y), (WIDTH, y), 1)

    pygame.draw.circle(screen, (20, 30, 55), CENTER, R_DISK_PX + 45, 3) # Cercle
    décoratif

# =====
# DESSIN DU DISQUE
# =====
def draw_disk(angle_rad):
    cx, cy = CENTER          # Centre du disque

    pygame.draw.circle(screen, (0, 0, 0), (cx + 10, cy + 12), R_DISK_PX) # Ombre
    pygame.draw.circle(screen, COPPER, (cx, cy), R_DISK_PX)              # Disque

    for i in range(0, R_DISK_PX, 20): # Cercles internes pour effet métallique
        color = (
            max(80, COPPER[0] - i // 3),
            max(45, COPPER[1] - i // 4),
            max(20, COPPER[2] - i // 5)
        )
        pygame.draw.circle(screen, color, (cx, cy), R_DISK_PX - i, 2)

    pygame.draw.circle(screen, COPPER_DARK, (cx, cy), R_DISK_PX, 8)      # Bord foncé
    pygame.draw.circle(screen, COPPER_LIGHT, (cx, cy), R_DISK_PX - 10, 2) # Bord clair

    pygame.draw.circle(screen, (30, 30, 30), (cx, cy), 18)              # Axe central
    pygame.draw.circle(screen, WHITE, (cx, cy), 18, 2)                  # Contour de l'axe

    L = R_DISK_PX - 20          # Longueur du rayon tournant

    x2 = cx + L * math.cos(angle_rad) # Coordonnée x du bout du rayon
    y2 = cy + L * math.sin(angle_rad) # Coordonnée y du bout du rayon

    pygame.draw.line(screen, WHITE, (cx, cy), (int(x2), int(y2)), 4)    # Rayon tournant
    pygame.draw.circle(screen, WHITE, (int(x2), int(y2)), 7)              # Point au bout

# =====
# DESSIN DU PION
# =====
def draw_pion(r_val):
    cx, cy = CENTER          # Centre du disque
    xw, yw = pion_world_pos(r_val) # Position du pion

    pygame.draw.line(screen, (220, 220, 220), (cx, cy), (int(xw), int(yw)), 4) # Bras
    gris
    pygame.draw.line(screen, BLUE, (cx, cy + 6), (int(xw), int(yw + 6)), 2)    # Bras
    bleu

```

```

    pygame.draw.circle(screen, (0, 0, 0), (int(xw + 4), int(yw + 5)), 15) # Ombre du
    pion
    pygame.draw.circle(screen, CARBON, (int(xw), int(yw)), 16) # Pion
    carbone
    pygame.draw.circle(screen, (100, 100, 100), (int(xw), int(yw)), 16, 3) # Contour
    pygame.draw.circle(screen, WHITE, (int(xw - 5), int(yw - 5)), 4) # Reflet

# =====
# TRACE ET HEATMAP
# =====
def update_trace(r_val):
    xw, yw = pion_world_pos(r_val) # Position du pion

    pygame.draw.circle(trace_surf, (255, 255, 255, 45), (int(xw), int(yw)), 3) # Trace
    blanche

    gx, gy = int(xw // 5), int(yw // 5) # Réduction de résolution pour la heatmap
    key = (gx, gy) # Case visitée

    visit[key] = visit.get(key, 0) + 1 # Incrémente le nombre de passages

    alpha = min(210, 15 + visit[key] * 4) # Intensité rouge selon le nombre de passages

    pygame.draw.rect(
        heat_surf,
        (255, 60, 60, alpha),
        pygame.Rect(gx * 5, gy * 5, 5, 5)
    )

# =====
# PANNEAU D'INFORMATIONS
# =====
def draw_hud(r_val, omega_val, nbp_val):
    rayon_cm = px_to_cm(r_val) # Conversion du rayon en cm
    v_tan_cm_s = omega_val * rayon_cm # Calcul de la vitesse tangentielle

    panel_x = 860 # Position x du panneau décalée
    panel_y = 30 # Position y du panneau
    panel_w = 500 # Largeur du panneau augmentée
    panel_h = 390 # Hauteur du panneau

    panel_rect = pygame.Rect(panel_x, panel_y, panel_w, panel_h)

    pygame.draw.rect(screen, PANEL_BG, panel_rect, border_radius=18) # Fond du
    panneau
    pygame.draw.rect(screen, (20, 40, 80), panel_rect, 4, border_radius=18) # Bordure

    title = FONT_BIG.render("BANC SLICE", True, (10, 25, 60)) # Titre
    screen.blit(title, (panel_x + 55, panel_y + 40)) # Affichage du titre

    subtitle = FONT_SMALL.render("Grandeurs en cm", True, GRAY) # Sous-titre
    screen.blit(subtitle, (panel_x + 55, panel_y + 100)) # Affichage sous-titre

    lines = [
        f"Nbp : {nbp_val}",
        f"Rayon : {rayon_cm:.2f} cm",
        f"Vitesse angulaire : {omega_val:.3f} rad/s",
        f"Vt réelle : {v_tan_cm_s:.2f} cm/s",
        f"V0 cible : {V0_CM_S:.1f} cm/s"
    ]
    y = panel_y + 145 # Position verticale du premier texte
    for s in lines: # Affichage ligne par ligne
        surf = FONT.render(s, True, BLACK)
        screen.blit(surf, (panel_x + 55, y))
        y += 40

```



```

pygame.draw.line(screen, (20, 40, 80), (panel_x + 40, panel_y + 330),
                  (panel_x + panel_w - 40, panel_y + 330), 2) # Séparation

commandes = [
    "ESPACE : Pause",
    "UP/DOWN: Modifier Nbp",
    "ESC    : Quitter"
]
y = panel_y + 345          # Position des commandes
for cmd in commandes:      # Affichage des commandes
    surf = FONT_SMALL.render(cmd, True, (120, 40, 20))
    screen.blit(surf, (panel_x + 55, y))
    y += 22

# =====
# BARRE DE VITESSE ANGULAIRE
# =====
def draw_speed_bar(omega_val):
    bar_x = 860             # Position x de la barre décalée
    bar_y = 455             # Position y de la barre
    bar_w = 500             # Largeur de la barre augmentée
    bar_h = 38              # Hauteur

    value = min(1.0, omega_val / 10) # Normalisation pour la largeur de la barre

    txt = FONT_SMALL.render("Niveau de vitesse angulaire", True, WHITE)
    screen.blit(txt, (bar_x, bar_y - 32))

    pygame.draw.rect(screen, PANEL_BG, (bar_x, bar_y, bar_w, bar_h), border_radius=15)
    pygame.draw.rect(screen, (20, 40, 80), (bar_x, bar_y, bar_w, bar_h), 3,
border_radius=15)

    pygame.draw.rect(
        screen,
        BLUE,
        (bar_x + 4, bar_y + 4, int((bar_w - 8) * value), bar_h - 8),
        border_radius=12
    )

# =====
# GRAPHIQUE DES COURBES
# =====
def draw_graph():
    graph_x = 860           # Position x du graphe décalée
    graph_y = 535           # Position y du graphe
    graph_w = 500           # Largeur du graphe augmentée
    graph_h = 320           # Hauteur du graphe

    pygame.draw.rect(screen, PANEL_BG, (graph_x, graph_y, graph_w, graph_h),
border_radius=15)
    pygame.draw.rect(screen, (20, 40, 80), (graph_x, graph_y, graph_w, graph_h), 3,
border_radius=15)

    title = FONT_SMALL.render("Courbes en temps reel", True, BLACK)
    screen.blit(title, (graph_x + 18, graph_y + 12))
    legend_y = graph_y + 42 # Position de la légende
    pygame.draw.circle(screen, RED, (graph_x + 22, legend_y), 6)
    screen.blit(FONT_SMALL.render("Rayon", True, BLACK), (graph_x + 35, legend_y - 9))

    pygame.draw.circle(screen, BLUE, (graph_x + 135, legend_y), 6)
    screen.blit(FONT_SMALL.render("Vitesse angulaire", True, BLACK), (graph_x + 148,
legend_y - 9))

    pygame.draw.circle(screen, GREEN, (graph_x + 355, legend_y), 6)
    screen.blit(FONT_SMALL.render("Vt", True, BLACK), (graph_x + 368, legend_y - 9))

```

```

plot_x = graph_x + 60          # Début x de la zone de tracé décalé à l'intérieur
plot_y = graph_y + 75          # Début y de la zone de tracé
plot_w = graph_w - 100         # Largeur zone de tracé ajustée
plot_h = graph_h - 110         # Hauteur zone de tracé

pygame.draw.line(screen, BLACK, (plot_x, plot_y + plot_h),
                  (plot_x + plot_w, plot_y + plot_h), 2) # Axe horizontal
pygame.draw.line(screen, BLACK, (plot_x, plot_y),
                  (plot_x, plot_y + plot_h), 2)          # Axe vertical

for i in range(1, 5):          # Grille horizontale
    y = plot_y + int(i * plot_h / 5)
    pygame.draw.line(screen, (200, 200, 200), (plot_x, y), (plot_x + plot_w, y), 1)

def scale_points(data, min_val, max_val):
    points = []                 # Liste des points convertis

    if len(data) < 2:           # Il faut au moins deux points
        return points

    for i, val in enumerate(data):
        x = plot_x + int(i * plot_w / (MAX_POINTS - 1))
        y = plot_y + plot_h - int((val - min_val) * plot_h / (max_val - min_val))
        y = max(plot_y, min(plot_y + plot_h, y))
        points.append((x, y))
    return points

if len(rayon_data) > 2:         # Tracé seulement si assez de points
    points_rayon = scale_points(rayon_data, RAYON_MIN_CM, RAYON_MAX_CM)
    points_omega = scale_points(omega_data, 0, 20)
    points_vt = scale_points(vt_data, 0, V0_CM_S * 1.2)

    pygame.draw.lines(screen, RED, False, points_rayon, 2)
    pygame.draw.lines(screen, BLUE, False, points_omega, 2)
    pygame.draw.lines(screen, GREEN, False, points_vt, 2)

# =====
# BOUCLE PRINCIPALE
# =====
while True:
    dt = clock.tick(FPS) / 1000.0 # Temps écoulé entre deux images
    for event in pygame.event.get(): # Lecture des événements
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                pygame.quit()
                sys.exit()
            if event.key == pygame.K_SPACE:
                paused = not paused

    keys = pygame.key.get_pressed() # Lecture continue du clavier
    if keys[pygame.K_UP]:
        Nbp += 1                     # Augmente Nbp
    if keys[pygame.K_DOWN]:
        Nbp = max(1, Nbp - 1)        # Diminue Nbp sans descendre sous 1
    dx = (Nbp * Rp) / 360            # Vitesse radiale du pion

    if not paused:
        r_px += r_dir * dx * dt      # Déplacement du pion
        if r_px >= r_max_px:
            r_px = float(r_max_px)
            r_dir = -1.0
        elif r_px <= r_min_px:
            r_px = float(r_min_px)

```

```

        r_dir = 1.0
        rayon_cm = px_to_cm(r_px)      # Rayon actuel en cm
        omega = V0_CM_S / max(rayon_cm, 1e-6) # Omega = Vt / r
        vt = omega * rayon_cm          # Vitesse tangentielle réelle
        theta = (theta + omega * dt) % (2 * math.pi) # Rotation du disque
        update_trace(r_px)             # Mise à jour de la trace
        rayon_data.append(rayon_cm)    # Ajout du rayon dans la courbe
        omega_data.append(omega)       # Ajout de omega
        vt_data.append(vt)             # Ajout de Vt

    if len(rayon_data) > MAX_POINTS:
        rayon_data.pop(0)
        omega_data.pop(0)
        vt_data.pop(0)
    else:
        rayon_cm = px_to_cm(r_px)
        omega = V0_CM_S / max(rayon_cm, 1e-6)

    draw_background()                # Dessin du fond
    screen.blit(heat_surf, (0, 0))  # Affichage heatmap
    screen.blit(trace_surf, (0, 0))  # Affichage trace
    draw_disk(theta)                 # Dessin disque
    draw_pion(r_px)                  # Dessin pion
    draw_hud(r_px, omega, Nbp)       # Affichage informations
    draw_speed_bar(omega)            # Barre vitesse angulaire
    draw_graph()                     # Courbes

    if paused:
        pause_text = FONT_BIG.render("PAUSE", True, YELLOW)
        screen.blit(pause_text, (WIDTH // 2 - 60, HEIGHT - 80))
    pygame.display.flip()           # Rafraîchissement écran

```